

# Relatório: Otimização de Footprint de Código Intermediário (LLVM-IR) via Compressão de Huffman

Disciplina: Análise de Algoritmos

Autores: Luiz Gustavo e Hendrick Silva

## 1. Introdução

A crescente prevalência de sistemas de computação de borda (*Edge Computing*) impõe desafios rigorosos à gestão de recursos computacionais, especificamente no que tange às restrições de memória *Flash* e *RAM*. Em cenários onde atualizações de firmware *Over-the-Air* (OTA) são necessárias via redes de banda limitada (como LoRaWAN ou NB-IoT), a redução do *footprint* de binários torna-se crítica para a eficiência energética e a redução do tempo de transmissão. Este trabalho apresenta uma solução de compressão sem perdas para arquivos de Código Intermediário LLVM-IR (.ll), utilizando o algoritmo de Codificação de Huffman implementado integralmente a partir do zero, sem o auxílio de contêineres da biblioteca padrão, garantindo controle total sobre a gestão de memória e o desempenho em sistemas restritos.

## 2. Modelagem do problema

O problema consiste em minimizar o comprimento total de uma sequência de símbolos  $S = \{s_1, s_2, \dots, s_k\}$  representando o alfabeto do código LLVM-IR. Buscamos o mapeamento binário  $c(s)$  que minimize:

$$\text{minimizar} \quad L(P) = \sum_{s \in S} f(s) \cdot |c(s)|$$

**Sujeito à restrição de código de prefixo:** nenhum  $c(s_i)$  pode ser prefixo de  $c(s_j)$  para  $i \neq j$ . Esta propriedade garante decodificação unívoca e instantânea, essencial para a eficiência do fluxo de bits. A granularidade do símbolo foi escolhida para equilibrar a representatividade do LLVM-IR e o custo do dicionário serializado.

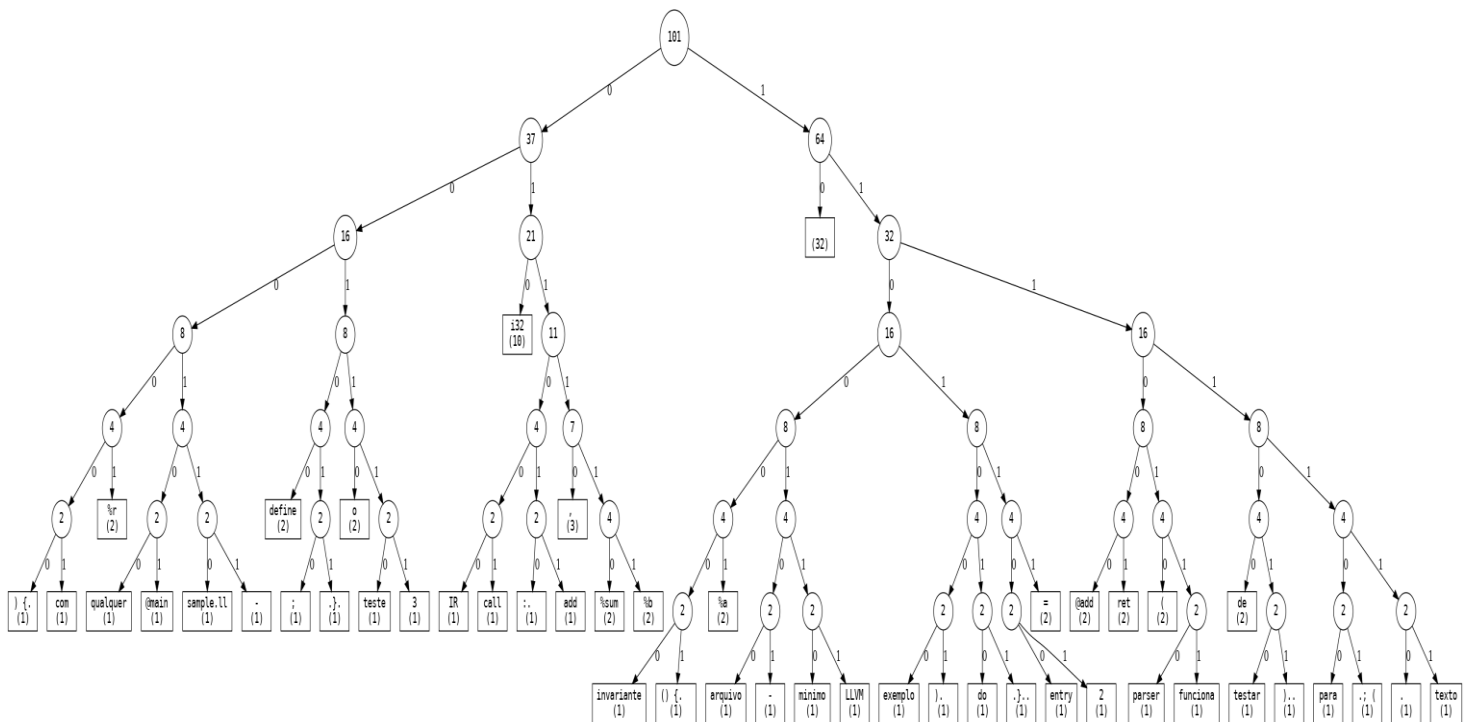
## 3. Arquitetura e implementação

O pipeline segue um fluxo de: *tokenização* → contagem de frequências → construção da árvore → geração de códigos → serialização.

- **Tokenização:** O analisador percorre o arquivo byte a byte e classifica cada caractere em duas categorias: *palavra* (letras, dígitos e símbolos de identificador do LLVM-IR, como `_`, `.`, `%`, `@`) ou *outro*. Em seguida, agrupa sequências máximas da mesma categoria em um único token. Essa

escolha corresponde à granularidade de símbolo definida na modelagem do problema e preserva a estrutura textual do LLVM-IR. O ponto central é que nenhum byte é descartado ou alterado: comentários, espaços, quebras de linha e pontuação também se tornam tokens. Portanto, concatenar todos os tokens, na mesma ordem, reproduz o arquivo original byte a byte, estabelecendo a invariante necessária para que a compressão seja reversível e sem perdas.

- **Gestão de Dados:** A contagem de frequências utiliza uma *hash table* estática ( $m=1024$ ) com encadeamento separado. A complexidade de inserção é  $O(1)$  média, sob a premissa de fator de carga  $\alpha < 1$ , validada experimentalmente por termos  $k \ll 1024$ .
- **Estruturas customizadas — min-heap:** Além da tabela de espalhamento, o projeto implementa do zero um min-heap binário sobre array, responsável por manter os nós da árvore ordenados pela frequência. A estrutura oferece inserção e extração do mínimo em  $O(\log k)$  por meio das operações de *sift-up* e *sift-down*. Esse componente viabiliza diretamente o algoritmo guloso de Huffman: a cada iteração, os dois nós de menor frequência são removidos, fundidos em um nó interno e reinseridos no heap. Assim, a hash table e o min-heap formam as duas estruturas próprias exigidas pelo enunciado, sem uso de contêineres da biblioteca padrão.
- **Serialização:** A árvore é serializada em pré-ordem. Dada a natureza de "árvore binária cheia" da codificação de Huffman (aridade fixa igual a 2), a reconstrução é autodelimitada utilizando apenas um *byte* de tag (1=folha, 0=interno) por nó, sem delimitadores adicionais.
- **Formato do arquivo comprimido:** O arquivo gerado é composto por uma assinatura *HUF1* de 4 bytes, seguida pelo número de tokens em *u32*, pelo tamanho do dicionário também em *u32*, pelo dicionário serializado e, por fim, pelo fluxo de bits comprimido. Como os códigos de Huffman possuem comprimento variável, o módulo de E/S de bits empacota os bits em bytes, do bit mais significativo para o menos significativo, completando o último byte com padding quando necessário. Na descompressão, o cabeçalho indica quantos tokens devem ser reconstruídos: primeiro a árvore é remontada pela desserialização e, depois, o fluxo de bits é percorrido até emitir exatamente a quantidade de tokens registrada.
- **Árvore de Huffman gerada para sample.ll:**



- **Garantia de reversibilidade (lossless):** O decodificador reconstrói o arquivo `.ll` original byte a byte. Essa propriedade foi validada por testes de *round-trip* — compactar, descompactar e comparar o resultado com o arquivo de entrada por *diff*. Em todos os cinco arquivos avaliados, a comparação resultou em arquivos idênticos, conforme indicado pela coluna *lossless = sim* na Tabela 1. Essa é a principal afirmação de corretude prática do sistema: embora o arquivo compactado altere a representação, a informação original é preservada integralmente.

#### 4. Análise de complexidade

O custo computacional segue a eficiência das estruturas customizadas implementadas:

| Função                                    | Tempo                  | Observação                                       |
|-------------------------------------------|------------------------|--------------------------------------------------|
| <b><i>parser_tokenize</i></b>             | <b><i>O(n)</i></b>     | Varredura única; cada byte classificado uma vez. |
| <b><i>ft_add</i></b>                      | <b><i>O(1)</i></b>     | Premissa de $\alpha < 1$ e hash uniforme.        |
| <b><i>mh_push</i> / <i>mh_pop_min</i></b> | <b><i>O(logk)</i></b>  | Sobe/desce a altura do min-heap.                 |
| <b><i>huff_build_tree</i></b>             | <b><i>O(klogk)</i></b> | Uso de min-heap em array.                        |
| <b><i>codes_build</i></b>                 | <b><i>O(k)</i></b>     | DFS sobre os $2k-1$ nós da                       |

| Função                                   | Tempo          | Observação                                 |
|------------------------------------------|----------------|--------------------------------------------|
|                                          |                | árvore.                                    |
| <i>dict_serialize / dict_deserialize</i> | $O(k)$         | Um passo por nó, em pré-ordem.             |
| <i>encode</i>                            | $O(n \cdot k)$ | Busca linear por símbolo.                  |
| <i>decode</i>                            | $O(B)$         | Um passo por bit (onde B é total de bits). |

A compressão é dominada pelo **encode**, ficando  $O(n \cdot k)$  no total (ou  $O(n + k \log k)$  com a otimização do hash), e a descompressão é  $O(k + B)$ .

## 5. Corretude e otimalidade

O algoritmo guloso garante a otimalidade via dois lemas fundamentais:

- **Lema 1 (Escolha Gulosa):** Se  $x$  e  $y$  são os símbolos de menor frequência, existe uma árvore ótima na qual  $x$  e  $y$  são folhas irmãs no nível mais profundo. Para demonstrar, considere uma árvore ótima e sejam  $a$  e  $b$  duas folhas mais profundas e irmãs. Como  $x$  tem frequência mínima,  $f(x) \leq f(a)$ . Ao trocar  $x$  com  $a$ , a variação no custo é  $\Delta = (f(x) - f(a)) \cdot (d_a - d_x)$ , em que  $d_a \geq d_x$  são as profundidades. Como  $f(x) - f(a) \leq 0$  e  $d_a - d_x \geq 0$ , segue que  $\Delta \leq 0$ ; portanto, a troca não aumenta o custo. Repetindo o argumento para  $y$  e  $b$ , conclui-se que existe uma árvore ótima com  $x$  e  $y$  como folhas irmãs no nível mais profundo.
- **Lema 2 (Subestrutura Ótima):** A fusão de  $x$  e  $y$  em um novo símbolo  $z$  com frequência  $f(z) = f(x) + f(y)$  reduz o problema para um alfabeto menor. Uma solução ótima para o alfabeto reduzido, ao expandir  $z$  de volta para  $x$  e  $y$ , permanece ótima para o original.
- **Conclusão:** Por indução no tamanho do alfabeto, o algoritmo guloso produz a árvore de custo mínimo.

## 6. Resultados experimentais

### 6.1 Metodologia

**Ambiente de teste.** Os experimentos foram executados em um notebook (hostname *Inspiron-15-3511*) com processador **Intel Core i3-1115G4 de 11ª geração (@ 3,00 GHz)** e **7,5 GiB de RAM**, sob **Linux Mint 21.1** (base Ubuntu 22.04). Todo o código foi compilado com **gcc 11.4.0**, usando as flags `-O2 -std=c11 -Wall -Wextra`. A ferramenta executa em thread única; os tempos reportados são a média de 5 execuções para reduzir o ruído de medição, sobretudo nos arquivos menores.

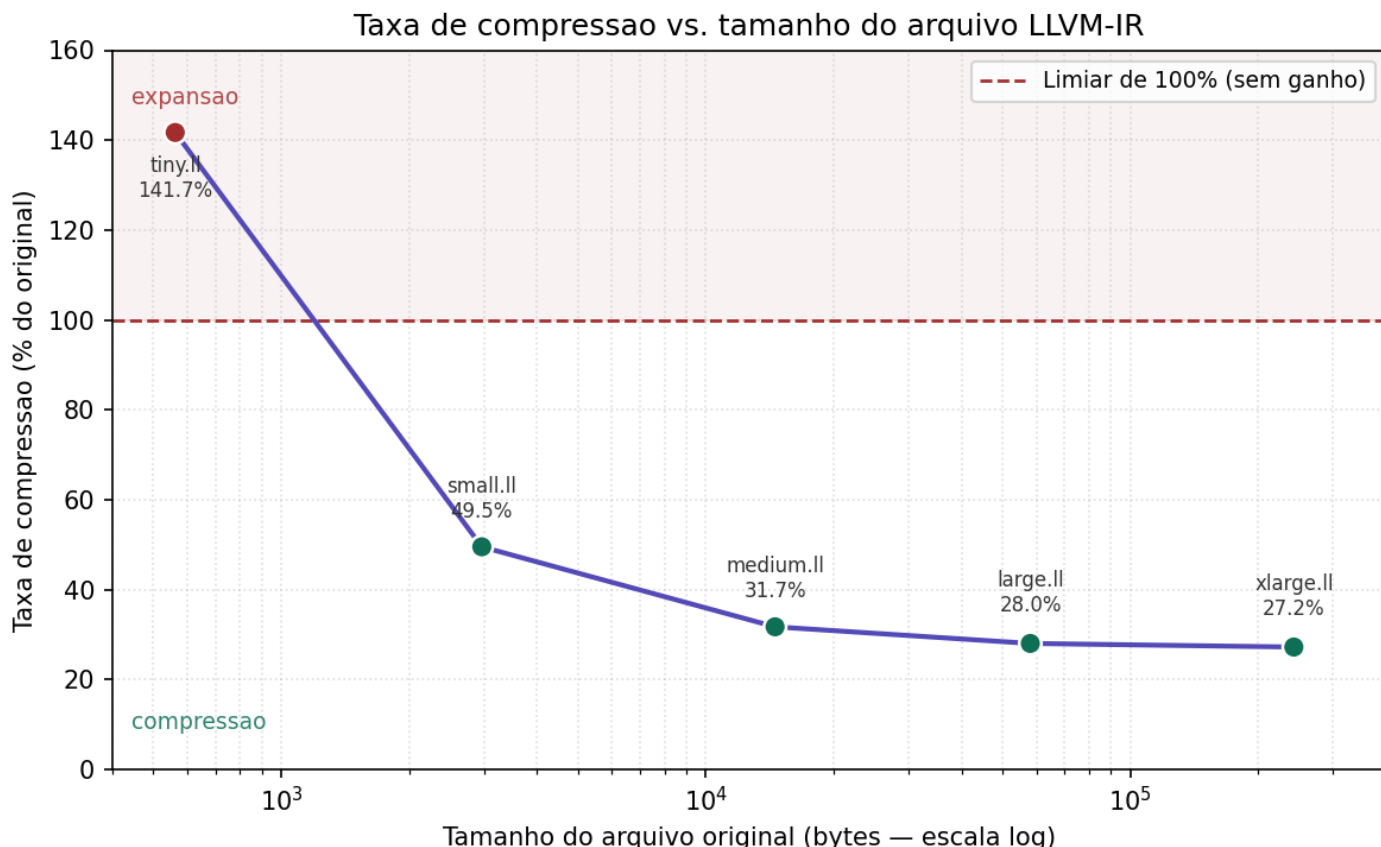
### 6.2 Dados coletados

Tabela 1 — Resultados de compressão (tempos = média de 5 execuções)

| arquivo   | original<br>(B) | comprimido<br>(B) | taxa<br>(%) | dicionário<br>(B) | t_comp<br>(s) | t_dec<br>(s) | lossless |
|-----------|-----------------|-------------------|-------------|-------------------|---------------|--------------|----------|
| tiny.ll   | 563             | 798               | 141.7       | 640               | 0.00098       | 0.00090      | sim      |
| small.ll  | 2 968           | 1 470             | 49.5        | 733               | 0.00111       | 0.00085      | sim      |
| medium.ll | 14 558          | 4 610             | 31.7        | 1 111             | 0.00140       | 0.00101      | sim      |
| large.ll  | 58 089          | 16 258            | 28.0        | 2 392             | 0.00515       | 0.00140      | sim      |
| xlarge.ll | 242 476         | 65 901            | 27.2        | 8 092             | 0.01284       | 0.00330      | sim      |

**Figura 1** — Taxa de compressão em função do tamanho do arquivo (escala log). A linha tracejada marca o limiar de 100% (sem ganho); pontos acima representam expansão, abaixo, compressão. O ponto de virada ocorre por volta de 1 KB, e a taxa converge para ~27% nos arquivos maiores.

### 6.3 Análise dos resultados



1. **Ponto de virada:** O arquivo tiny.ll apresentou expansão (141,7%), demonstrando que para arquivos abaixo de ~1 KB, o custo do dicionário supera o benefício da compressão.
2. **Taxa assintótica:** Observamos convergência em torno de 27% para arquivos maiores (large e xlarge), definindo o limite de eficiência desta abordagem.
3. **Diluição do dicionário:** O peso do dicionário cai de 80% (tiny.ll) para 12% (xlarge.ll), evidenciando que a compressão se torna altamente vantajosa à medida que o tamanho do arquivo LLVM-IR cresce.
4. **Desempenho:** A descompressão foi consistentemente mais rápida que a compressão em todos os casos testados, e a diferença torna-se mais perceptível nos arquivos maiores. Esse comportamento confirma que o custo principal está concentrado na etapa de compressão, especialmente na busca dos códigos e na construção das estruturas auxiliares.

## 7. Conclusão e trabalhos futuros

Este trabalho apresentou uma implementação completa de compressão sem perdas para arquivos LLVM-IR utilizando Codificação de Huffman construída do zero. A reversibilidade foi confirmada em todos os arquivos avaliados por meio de testes de round-trip, nos quais o arquivo compactado foi descompactado e comparado com o original, resultando em saídas idênticas. Os experimentos também quantificaram o trade-off entre o custo do dicionário e o tamanho da entrada: em arquivos muito pequenos, como tiny.ll, houve expansão devido ao peso fixo do dicionário; a partir de aproximadamente 1 KB, a compressão passou a ser vantajosa; e, nos arquivos maiores, a taxa convergiu para cerca de 27% do tamanho original.

Como trabalhos futuros, uma extensão natural é ampliar a granularidade dos símbolos para n-grams, isto é, sequências de instruções ou padrões recorrentes do LLVM-IR, o que pode melhorar a compressão, mas também aumenta o tamanho do dicionário. Outra possibilidade é formular a escolha dessas sequências como uma variação do Problema da Mochila, usando Programação Dinâmica para decidir quais padrões devem virar tokens considerando ganho de compressão e custo de armazenamento. Além disso, há otimizações diretas na implementação, como reduzir o campo de tamanho dos símbolos no dicionário para 1 byte quando aplicável e substituir a busca linear símbolo→código por uma tabela hash, levando o encode de  $O(n \cdot k)$  para  $O(n)$  em média.

## Referências

HUFFMAN, David A. A method for the construction of minimum-redundancy codes. Proceedings of the IRE, New York, v. 40, n. 9, p. 1098-1101, 1952.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

LLVM PROJECT. LLVM Language Reference Manual. Disponível em: <<https://llvm.org/docs/LangRef.html>>. Acesso em: 11 jun. 2026.